

DATA STRUCTURES (BCSE-007)

Official Comprehensive Master Lecture Notes & Curriculum Compendium

Institution: Maharishi Markandeshwar (Deemed to be University) - MMEC **Program:** B.Tech (2nd Semester - All Streams) **Course Code:** BCSE-007

Scheme: L:2 T:0 P:0 | Credits: 2.0 **Total Marks:** 100 (40 Internal + 60 End-Sem)

UNIT I: INTRODUCTION, ARRAYS, SEARCHING & SORTING

PART 1 • FOUNDATIONS & COMPLEXITY

1.1 Basic Terminologies & Elementary Data Organization

A **Data Structure** is a specialized format for organizing, storing, and retrieving data efficiently. The choice of structure directly reflects the mathematical and logical relationships between real-world items:

- **Data Item:** A single elementary value (e.g., Student Age, Roll Number).
- **Entity:** An object possessing concrete attributes (e.g., an Employee or Student).
- **Record:** A collection of related data items representing an entity instance.
- **File:** A collection of logically related records (e.g., University_Registry table).
- **Primary Key:** An attribute that uniquely distinguishes each record in a file.

1.2 Linear vs. Non-Linear Data Structures

Feature	Linear Data Structures	Non-Linear Data Structures
Organization	Sequential; each element has unique predecessor & successor.	Hierarchical or interconnected multi-level network.
Traversal	Single linear pass visits all elements sequentially.	Multiple traversal paths (depth, breadth, level-order).
Memory Layout	Contiguous (Arrays) or linear pointer-linked (Linked Lists).	Non-contiguous memory with multiple branch pointers.
Examples	Arrays, Stacks (LIFO), Queues (FIFO), Singly/Doubly Linked Lists.	Trees (Binary, BST, AVL), Graphs (Directed, Undirected).

1.3 Fundamental Data Structure Operations

1. Traversing: Visiting every element exactly once. • **2. Searching:** Locating an element by key. • **3. Insertion:** Adding a new item at a valid position. • **4. Deletion:** Removing an element and adjusting capacity. • **5. Sorting:** Ordering elements in ascending/descending sequence. • **6. Merging:** Combining two sorted collections into one.

1.4 Algorithm Analysis & Time-Space Trade-off

Time Complexity $T(n)$: Total elementary operations executed as a function of input size n .

Space Complexity $S(n)$: Total memory consumed: Fixed (code, constants) + Variable (heap, stack).

Time-Space Trade-off: Memory can be traded for execution speed (e.g., Fast Transpose of Sparse Matrix taking $O(C+K)$ time using extra auxiliary memory vs Simple Transpose taking $O(C \times K)$ with $O(1)$ space).

1.5 Asymptotic Notations & Asymptotic Growth Hierarchy

Notation	Formal Mathematical Definition	Intuitive Bound
Big-Oh [O]	$f(n) \leq c \cdot g(n)$ for all $n \geq n_0$ ($c > 0, n_0 \geq 1$)	Upper Bound: Worst-case execution ceiling.
Big-Omega [Ω]	$f(n) \geq c \cdot g(n)$ for all $n \geq n_0$ ($c > 0, n_0 \geq 1$)	Lower Bound: Best-case execution floor.
Big-Theta [Θ]	$c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$	Tight Bound: Asymptotically exact performance.

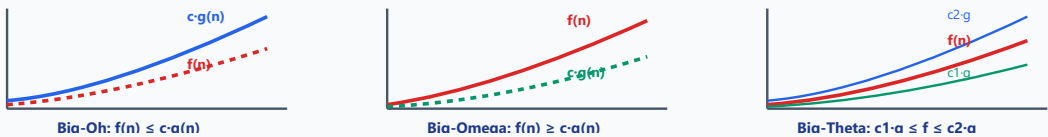


FIGURE 1.1: GEOMETRIC VISUALIZATION OF ASYMPTOTIC UPPER, LOWER, AND TIGHT BOUNDS

Growth Hierarchy: $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

1.6 Linear Arrays (1D Array) & Memory Representation

A **Linear Array** is a homogeneous collection of data elements stored in contiguous physical memory locations. Any element is directly accessible in $O(1)$ time via its mathematical index offset.

1D Array Address Formula: $\text{Loc}(A[k]) = \text{Base}(A) + w \times (k - \text{LB})$

Where: $\text{Base}(A)$ = starting address; w = element size in bytes; LB = Lower Bound index ($\emptyset$ in C); k = target index.

1.7 2D Arrays: Row-Major Order (RMO) vs. Column-Major Order (CMO)

Because computer memory hardware is strictly one-dimensional (linear byte addresses), a 2D matrix $A[M][N]$ (M rows, N columns) must be linearized:

Row-Major Order (RMO) [C, C++, Java, Python]:

Stores row 0 contiguously, then row 1, then row 2...

Formula: $\text{Loc}(A[i][j]) = \text{Base}(A) + w \times [(i - \text{LB}_r) \times N_c + (j - \text{LB}_c)]$

Where: N_c = Total columns = $(\text{UB}_c - \text{LB}_c + 1)$.

Column-Major Order (CMO) [Fortran, MATLAB, R]:

Stores column 0 contiguously, then column 1, then column 2...

Formula: $\text{Loc}(A[i][j]) = \text{Base}(A) + w \times [(j - \text{LB}_c) \times N_r + (i - \text{LB}_r)]$

Where: N_r = Total rows = $(\text{UB}_r - \text{LB}_r + 1)$.

2D Matrix $A[2][3]$:

$[0][0]$	$[0][1]$	$[0][2]$
$[1][0]$	$[1][1]$	$[1][2]$

Row-Major Order (RMO) in RAM:

Row 0: $[0][0]$	Row 0: $[0][1]$	Row 0: $[0][2]$	Row 1: $[1][0]$	Row 1: $[1][1]$	Row 1: $[1][2]$
-----------------	-----------------	-----------------	-----------------	-----------------	-----------------

→ All elements of Row 0 stored before Row 1 begins.

FIGURE 1.2: RAM LAYOUT LINEARIZATION: ROW-MAJOR ORDER SEQUENTIAL STORAGE

High-Yield Numerical on 2D Array Addressing:

Problem: Given an array $A[2..8, -1..5]$ with Base Address = 2000, and element size $w = 4$ bytes. Compute $\text{Loc}(A[5][2])$ under Row-Major and Column-Major orders.

Solution:

$\text{LB}_r = 2, \text{UB}_r = 8 \Rightarrow N_r = 8 - 2 + 1 = 7$ rows.

$\text{LB}_c = -1, \text{UB}_c = 5 \Rightarrow N_c = 5 - (-1) + 1 = 7$ columns.

Target: $i = 5, j = 2 \Rightarrow (i - \text{LB}_r) = 3; (j - \text{LB}_c) = 3$.

Row-Major Order (RMO):

$\text{Loc}(A[5][2]) = 2000 + 4 \times [3 \times 7 + 3] = 2000 + 4 \times [21 + 3] = 2000 + 4 \times 24 = \mathbf{2096}$.

Column-Major Order (CMO):

$\text{Loc}(A[5][2]) = 2000 + 4 \times [3 \times 7 + 3] = 2000 + 4 \times [21 + 3] = \mathbf{2096}$. (Equal here because $N_r = N_c$).

1.8 Multi-Dimensional Array Generalization (3D Array)

For a 3D array $A[1..D_1, 1..D_2, 1..D_3]$ stored in Row-Major order:

$\text{Loc}(A[i][j][k]) = \text{Base}(A) + w \times [(i - \text{LB}_1) \times D_2 \times D_3 + (j - \text{LB}_2) \times D_3 + (k - \text{LB}_3)]$

1.9 Sparse Matrix & 3-Tuple (Triplet) Representation

A **Sparse Matrix** is a matrix where the vast majority of elements are zeros. Storing it as a standard 2D array wastes memory and processor cycles performing trivial arithmetic on zeros.

3-Tuple (Triplet) Representation:

Stored in a 2D array $S[K+1][3]$, where K = count of non-zeroes:

- **Row 0 (Header):** [Total Rows, Total Cols, Total Non-Zeroes (K)]
- **Rows 1 to K :** [Row Index, Col Index, Value]

Compression: For a 1000×1000 matrix with 500 non-zeroes: standard storage takes 4 MB; Triplet takes ~6 KB (99.85% savings!).

Sparse Matrix Transpose Algorithms:

1. **Simple Transpose:** Scans triplet K times for each column 0 to $C-1$.

Time Complexity: $O(C \times K)$. Space: $O(K)$.

2. **Fast Transpose:** Uses `total[col]` frequency array and `start[col]` index table to place elements directly in 1 pass.

Time Complexity: $O(C + K)$ – **Linear Time!**

1.10 Searching Techniques: Linear Search vs. Binary Search

Parameter	Linear Search (Sequential)	Binary Search (Divide & Conquer)
Prerequisite	None; works on unsorted, sorted, and linked lists.	Array must be sorted with $O(1)$ random indexing.
Search Principle	Sequentially scans every item from index 0 to $n-1$.	Compares key with midpoint; eliminates 50% search space per step.
Recurrence	$T(n) = T(n-1) + c$	$T(n) = T(n/2) + c \Rightarrow$ Solves to $O(\log_2 n)$ via Master Theorem.
Best Case Time	$O(1)$ — Target key is located at index 0.	$O(1)$ — Target key is located at middle index.
Worst / Avg Time	$O(n)$ — Key at end or absent.	$O(\log_2 n)$ — Halves interval each step.
Space Complexity	$O(1)$ auxiliary space.	$O(1)$ iterative; $O(\log n)$ recursive (call stack frames).

```
// Binary Search in C (Iterative)
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // Prevents overflow!
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // Not found
}
```

Key Engineering Insight on Midpoint:

Using `mid = (low + high) / 2` can cause signed 32-bit integer overflow if `low + high > 2,147,483,647`. Always write:

`mid = low + (high - low) / 2`

This guarantees mathematical accuracy without integer overflow!

Why Binary Search Fails on Linked Lists: In a linked list, finding the middle element takes $O(n)$ sequential traversal time. Repeating this gives $O(n) + O(n/2) + \dots = O(n)$, completely eliminating the $O(\log n)$ logarithmic advantage of binary search!

1.11 Comprehensive Sorting Algorithms

A sorting algorithm is **Stable** if it preserves the original relative order of duplicate keys; it is **In-Place** if it requires $O(1)$ auxiliary space beyond the input array.

1. Selection Sort

Scans unsorted portion to find minimum key, then swaps it to current index. Exactly $n-1$ passes and $n(n-1)/2$ comparisons. **Time: $\Theta(n^2)$ in all cases. Space: $O(1)$. Unstable.**

2. Bubble Sort

Compares adjacent pairs and swaps if out of order, bubbling heaviest element to end. With a swapped boolean flag, best-case is **$O(n)$** . Worst/Avg: **$O(n^2)$. Stable. In-Place.**

3. Insertion Sort

Takes current element and inserts it into sorted prefix (like playing cards). Best Case: **$O(n)$** (already sorted). Worst Case: **$O(n^2)$. Stable. Online. Adaptive.**

4. Merge Sort (Divide & Conquer)

Divides array at mid, sorts halves recursively, merges sorted halves using two pointers. Recurrence: $T(n) = 2T(n/2) + \Theta(n)$. **Time: $\Theta(n \log n)$ all cases. Space: $O(n)$. Stable.**

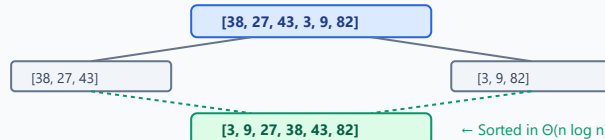


FIGURE 1.3: MERGE SORT DIVIDE & CONQUER TREE DECOMPOSITION AND MERGING

1.12 Master Sorting Complexity Matrix

Algorithm	Best Case Time	Average Case Time	Worst Case Time	Aux Space	Stability	Core Algorithmic Technique
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	No (Unstable)	Selection of minimum; exactly $n-1$ swaps.
Bubble Sort	$\Omega(n)$ [Flagged]	$\Theta(n^2)$	$O(n^2)$	$O(1)$	Yes (Stable)	Adjacent comparisons; early exit with flag.
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	Yes (Stable)	Incremental insertion; optimal for nearly sorted data.
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$	Yes (Stable)	Divide and conquer; guaranteed $\Theta(n \log n)$ upper bound.

Why MergeSort is Preferred for Linked Lists: Linked lists provide sequential access, making two-pointer merging $O(1)$ in extra memory (simply rearranging pointers without allocating auxiliary arrays), unlike array MergeSort which requires $O(n)$ extra space!

2.1 Stack Abstract Data Type (ADT) & LIFO Principle

A **Stack** is a linear collection of homogeneous elements operating strictly under the **LIFO (Last-In, First-Out)** principle. All insertions (push) and deletions (pop) occur at a single distinguished end called **TOP**.

Core Operations:

- **push(x)**: Inserts element at TOP. **Time: O(1)**.
- **pop()**: Removes TOP element. **Time: O(1)**.
- **peek()** / **top()**: Reads TOP value. **Time: O(1)**.
- **isEmpty()**: Returns $\text{top} == -1$. **Time: O(1)**.
- **isFull()**: Returns $\text{top} == \text{MAX} - 1$. **Time: O(1)**.

Stack Boundary Conditions:

1. **Stack Overflow**: Occurs when attempting to push onto a full stack ($\text{top} == \text{MAX} - 1$).
2. **Stack Underflow**: Occurs when attempting to pop from an empty stack ($\text{top} == -1$).

Stack Array Storage [MAX=5]:



- **Push(40)**: TOP advances to 3, $\text{arr}[3] = 40$
- **Pop()**: Returns $\text{arr}[2]$ (30), TOP decreases to 1
- All push/pop/peek operations run in $O(1)$ constant time

FIGURE 2.1: STACK ARRAY MEMORY REPRESENTATION AND TOP POINTER DISCIPLINE

2.2 Arithmetic Expression Notations

An expression consists of operands and operators. Three canonical notations exist in computing:

1. **Infix (Human Format)**: Operator between operands (e.g., $\backslash(A + B\backslash)$). Requires parentheses and precedence rules.
2. **Prefix (Polish Notation)**: Operator before operands (e.g., $\backslash(+ A B\backslash)$). Eliminates parentheses.
3. **Postfix (Reverse Polish Notation - RPN)**: Operator after operands (e.g., $\backslash(A B +\backslash)$). Eliminates parentheses and operator ambiguity; universal in compilers and stack bytecode machines.

2.3 Infix to Postfix Conversion (Shunting-Yard Rules)

Operator Precedence & Associativity:

- **Exponentiation (^)**: Precedence 3, Associativity: **Right-to-Left**
- **Multiplication / Division (*, /)**: Precedence 2, Associativity: **Left-to-Right**
- **Addition / Subtraction (+, -)**: Precedence 1, Associativity: **Left-to-Right**

Conversion Rules:

1. If operand $\rightarrow$ Output directly to Postfix string.
2. If '(' $\rightarrow$ Push onto Stack.
3. If ')' $\rightarrow$ Pop from Stack to Output until '(' is popped. Discard both parentheses.
4. If operator op: Pop operators from Stack to Output while stack is not empty and $\text{prec}(\text{stack}[\text{top}]) > \text{prec}(\text{op})$, OR $(\text{prec}(\text{stack}[\text{top}]) == \text{prec}(\text{op}) \text{ and } \text{op is Left-Associative})$. Then push op.
5. At input end $\rightarrow$ Pop all remaining operators from Stack to Output.

2.4 Complete Infix to Postfix Tracing Table

Tracing the expression: $((A + B) * C) - (D / E)$

Step	Scanned Token	Stack Status (Bottom → Top)	Postfix Output Generated	Action Taken & Rule Applied
1	(	(	—	Left parenthesis → Push to stack
2	(	((	—	Left parenthesis → Push to stack
3	A	((	A	Operand → Append directly to output
4	+	((+	A	Operator → Push '+' onto '('
5	B	((+	A B	Operand → Append directly to output
6	)	(	A B +	)' → Pop '+' until '('; discard pair
7	*	(*	A B +	Operator → Push '*' onto '('
8	C	(*	A B + C	Operand → Append to output
9	)	[empty]	A B + C *	)' → Pop '*' until '('; discard pair
10	-	-	A B + C *	Stack empty → Push '-'
11	(	-(	A B + C *	Left parenthesis → Push to stack
12	D	-(	A B + C * D	Operand → Append to output
13	/	-(/	A B + C * D	Operator → Push '/' onto '('
14	E	-(/	A B + C * D E	Operand → Append to output
15	)	-	A B + C * D E /	)' → Pop '/' until '('; discard pair
16	End	[empty]	A B + C * D E / -	End of input → Pop remaining '-' to output

2.5 Postfix Expression Evaluation Algorithm

Evaluating an RPN string requires a single stack of numeric values:

Algorithm:

1. Scan Postfix string token by token from left to right.
2. If token is a number → Push onto evaluation stack.
3. If token is an operator $\backslash(\backslash\text{odot}\backslash)$:
 - $\text{op2} = \text{pop}()$; $\text{op1} = \text{pop}()$;
 - $\text{result} = (\text{op1} \backslash(\backslash\text{odot}\backslash) \text{op2})$;
 - Push result back onto stack.
4. At end, top of stack holds the final scalar value.

Step-by-Step Numerical Evaluation Trace:

Evaluate Postfix Expression: 6 2 3 + * 12 4 / -

- Token 6 → Push(6) [Stack: 6]
- Token 2 → Push(2) [Stack: 6, 2]
- Token 3 → Push(3) [Stack: 6, 2, 3]
- Token '+' → $\text{op2}=3, \text{op1}=2 \rightarrow 2 + 3 = 5 \rightarrow \text{Push}(5)$ [Stack: 6, 5]
- Token '*' → $\text{op2}=5, \text{op1}=6 \rightarrow 6 * 5 = 30 \rightarrow \text{Push}(30)$ [Stack: 30]
- Token 12 → Push(12) [Stack: 30, 12]
- Token 4 → Push(4) [Stack: 30, 12, 4]
- Token '/' → $\text{op2}=4, \text{op1}=12 \rightarrow 12 / 4 = 3 \rightarrow \text{Push}(3)$ [Stack: 30, 3]
- Token '-' → $\text{op2}=3, \text{op1}=30 \rightarrow 30 - 3 = 27 \rightarrow \text{Push}(27)$ [Stack: 27]

Final Calculated Result: 27.

2.6 Principles of Recursion & System Call Stack

Recursion is a programming paradigm where a function calls itself directly or indirectly to solve smaller instances of the same problem. Every recursive function must possess:

1. Base Case (Anchor Condition):

A terminating condition that returns a direct value without further recursive calls. Prevents infinite recursion stack overflow.

2. Recursive Step (Inductive Step):

Reduces problem to smaller sub-instances that progress monotonically toward the base case.

Runtime Activation Records (Stack Frames)

Every function call allocates an **Activation Record (Stack Frame)** on the OS Call Stack storing: Return instruction address, Parameters, Local variables, and Saved CPU registers. Stack frames unwind in LIFO order when returning.



FIGURE 2.2: RUNTIME CALL STACK MECHANICS: WINDING (PUSH) & UNWINDING (POP)

2.7 Recursion vs. Iteration Comparative Matrix

Parameter	Recursion	Iteration
Mechanism	Function calls itself repeatedly until base condition holds.	Loop structures (for, while) execute repeatedly until test fails.
Space Overhead	O(n) auxiliary stack memory for activation records.	O(1) auxiliary space (registers / loop counters).
Speed / Efficiency	Slower due to function call overhead and stack allocation.	Faster; no stack allocation or context switching occurs.
Code Structure	Clean, compact, and mathematically elegant for divide-and-conquer.	Can require complex explicit stack management for trees/graphs.
Failure Mode	Stack Overflow if recursion depth exceeds RAM limit.	Infinite Loop if loop termination condition is never reached.

2.8 Types of Recursion

- **Tail Recursion:** Recursive call is the last statement executed. Can be optimized by compilers into a loop.
- **Head Recursion:** Recursive call is made before processing the rest of the function.
- **Tree Recursion:** Function makes two or more recursive calls per invocation (e.g., Fibonacci).
- **Indirect Recursion:** Function A calls Function B, which in turn calls Function A.

2.9 Fibonacci Numbers: Recursive vs. Iterative Analysis

Fibonacci recurrence: $F(0) = 0$, $F(1) = 1$, and $F(n) = F(n-1) + F(n-2)$ for $(n \geq 2)$.

```
// Recursive Fibonacci: O(2^n) Catastrophic!
int fibRecursive(int n) {
    if (n <= 0) return 0;
    if (n == 1) return 1;
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

// Iterative Fibonacci: O(n) Time, O(1) Space
int fibIterative(int n) {
    if (n <= 0) return 0;
    int a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) {
        c = a + b; a = b; b = c;
    }
    return b;
}
```

Recursive Redundancy Analysis:

Naive recursive Fibonacci constructs an exponential binary call tree. It evaluates identical subproblems repeatedly: computing $F(5)$ evaluates $F(3)$ two times and $F(2)$ three times!

- **Time Complexity:** $O(2^n)$ — Catastrophic blowup!
- **Space Complexity:** $O(n)$ stack height.
- **Iterative Solution:** $O(n)$ time, $O(1)$ memory.

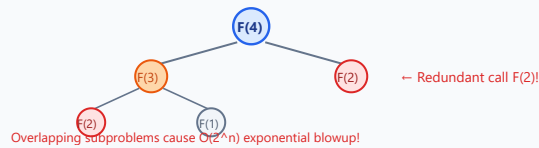


FIGURE 2.3: RECURSIVE CALL TREE FOR FIBONACCI DEMONSTRATING OVERLAPPING REDUNDANCY

2.10 Tower of Hanoi: Mathematical Recurrence & Proof

Governing Rules: 3 pegs (Source A, Aux B, Dest C), n distinct disks.

1. Only 1 disk moved at a time.
2. Upper disk moved to another peg.
3. No larger disk placed on smaller disk.

Recurrence Derivation:

Move $(n-1)$ disks $A \rightarrow B$ using C : $T(n-1)$
 Move disk n from $A \rightarrow C$: 1 move
 Move $(n-1)$ disks $B \rightarrow C$ using A : $T(n-1)$
 $T(n) = 2T(n-1) + 1$, with $T(1) = 1$
 $T(n) = 2[2T(n-2) + 1] + 1 = 2^2T(n-2) + 2 + 1$
 $T(n) = 2^n - 1$ moves.

```
// Tower of Hanoi C Implementation
void towerOfHanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", from, to);
        return;
    }
    towerOfHanoi(n - 1, from, aux, to);
    printf("Move disk %d from %c to %c\n", n, from, to);
    towerOfHanoi(n - 1, aux, to, from);
}
```

Complete Move Table for $n = 3$ Disks (Total Moves = $2^3 - 1 = 7$)

Move #	Disk Moved	From Peg	To Peg	Aux Peg	Status After Move
1	Disk 1 (Smallest)	A	C	B	A: [3, 2] B: [] C: [1]
2	Disk 2	A	B	C	A: [3] B: [2] C: [1]
3	Disk 1	C	B	A	A: [3] B: [2, 1] C: []
4	Disk 3 (Largest)	A	C	B	A: [] B: [2, 1] C: [3]
5	Disk 1	B	A	C	A: [1] B: [2] C: [3]
6	Disk 2	B	C	A	A: [1] B: [] C: [3, 2]
7	Disk 1	A	C	B	A: [] B: [] C: [3, 2, 1] (COMPLETED!)

3.1 Queue Abstract Data Type (ADT) & FIFO Discipline

A **Queue** is a linear data structure operating under the **FIFO (First-In, First-Out)** discipline. Elements are inserted at **REAR** and deleted from **FRONT**.

3.2 Types of Queues & Problem of False Overflow

1. Linear Queue Drawback (False Overflow / Memory Drift): In a linear array queue, as elements are dequeued, FRONT advances forward. Once $\text{rear} == \text{MAX} - 1$, no new elements can be enqueued even if slots 0 to FRONT-1 are completely vacant!

2. Circular Queue (Solution to Memory Drift):

Treats array as a continuous ring where index $\text{SIZE} - 1$ wraps around to 0 via **Modulo Arithmetic**:

- **Enqueue Pointer Advance:** $\text{rear} = (\text{rear} + 1) \% \text{SIZE}$
- **Dequeue Pointer Advance:** $\text{front} = (\text{front} + 1) \% \text{SIZE}$
- **Full Condition:** $(\text{rear} + 1) \% \text{SIZE} == \text{front}$
- **Empty Condition:** $\text{front} == -1$

Circular Queue Ring (SIZE = 6):



(Current dequeue item) **REAR** at index 5 (Wraps around to index 0!) Next Enqueue: $\text{rear} = (5 + 1) \% 6 = 0$ (Reuses Slot 0!) Full Check: $(\text{rear} + 1) \% \text{SIZE} == \text{front} \rightarrow$ Memory drift eliminated!

FIGURE 3.1: CIRCULAR QUEUE MODULO WRAP-AROUND RING ARCHITECTURE

3. Priority Queue

Elements have assigned priorities; deletion extracts the highest priority element regardless of arrival order.

- **Unsorted Array:** Insert $O(1)$, Delete $O(n)$.
- **Sorted Array:** Insert $O(n)$, Delete $O(1)$.
- **Binary Heap:** Insert $O(\log n)$, Delete $O(\log n)$ — Industry standard.

4. Real-World Applications of Queues

- **CPU Process Scheduling:** Round-Robin task queues.
- **I/O Buffers:** Print spooling, keyboard keystrokes.
- **Graph Algorithms:** Breadth-First Search (BFS).
- **Networking:** Packet router buffers.

3.3 Complete Circular Queue C Implementation

```
#define SIZE 5
int q[SIZE], front = -1, rear = -1;

int isFull() { return ((rear + 1) % SIZE == front); }
int isEmpty() { return (front == -1); }

void enqueue(int val) {
    if (isFull()) { printf("Queue Overflow!\n"); return; }
    if (isEmpty()) front = rear = 0;
    else rear = (rear + 1) % SIZE;
    q[rear] = val;
}

int dequeue() {
    if (isEmpty()) { printf("Queue Underflow!\n"); return -1; }
    int val = q[front];
    if (front == rear) front = rear = -1; // Reset when queue becomes empty
    else front = (front + 1) % SIZE;
    return val;
}
```

3.4 Dynamic Memory Allocation in C & Node Anatomy

A **Linked List** is a dynamic linear data structure consisting of nodes scattered in heap memory, chained together by explicit pointers. Dynamic functions: `malloc()`, `free()`, `calloc()`, `realloc()`.

Array vs. Linked List:

- **Array:** Fixed size, contiguous memory, $O(1)$ random index access, expensive $O(n)$ insertions/deletions.
- **Linked List:** Dynamic sizing, non-contiguous memory, $O(n)$ sequential access, $O(1)$ pointer-based insertions/deletions.

Singly Linked Node Structure in C:

```
struct Node {
    int data;           // 4 bytes data
    struct Node *next; // 8 bytes pointer
};
```

3.5 Singly Linked List (SLL) Operations & Complexity

Operation	Algorithm Pointer Updates	Time Complexity
Insert at Beginning	<code>newNode->next = head; head = newNode;</code>	$O(1)$
Insert at End	Traverse to last node (<code>curr->next == NULL</code>); <code>curr->next = newNode;</code>	$O(n)$ [$O(1)$ with tail]
Delete from Beginning	<code>temp = head; head = head->next; free(temp);</code>	$O(1)$
Delete from End	Traverse to second-last node; <code>prev->next = NULL; free(curr);</code>	$O(n)$
Search Element	Traverse sequentially checking <code>curr->data == key</code> until found or NULL.	$O(n)$

Singly Linked List (SLL) Pointer Architecture:



- Non-contiguous heap nodes chained via pointers.
- Insert at beginning is $O(1)$; No shifting required.

FIGURE 3.2: SINGLY LINKED LIST NODE ANATOMY AND TERMINATION AT NULL

3.6 Linked Representation of Stack & Queue

Linked Stack: Top is kept at head:

- `push(x)`: Insert at head → **$O(1)$** .
- `pop()`: Delete from head → **$O(1)$** .
- Never overflows; memory dynamically grows.

Linked Queue: Keeps front & rear pointers:

- `enqueue(x)`: Insert at `rear->next` → **$O(1)$** .
- `dequeue()`: Delete from front → **$O(1)$** .
- Eliminates false overflow completely without modulo!

3.7 Doubly Linked List (DLL) & Circular Linked List (CLL)

Doubly Linked List (DLL):

- Has prev and next pointers.
- Bidirectional traversal (forward & backward).
- Deleting a node given its pointer is **$O(1)$** without traversing to find predecessor!

Circular Linked List (CLL):

- Last node's next points back to head.
- No node points to NULL.
- Ideal for round-robin scheduling & circular buffers.

4.1 Tree Terminologies & Mathematical Properties

A **Tree** is a non-linear hierarchical data structure consisting of nodes connected by edges, containing no cycles.

- **Root:** Unique node with in-degree = 0. • **Leaf:** Node with no children (degree = 0).
- **Degree of Node:** Number of children. • **Height / Depth:** Longest path length from root to leaf.
- **Level:** Root is Level 0; children are Level 1, grandchildren Level 2, etc.

4.2 Binary Trees & Types

Binary Tree Type	Structural Definition	Mathematical Property
Full (Strict) BT	Every node has either 0 or 2 children (no degree-1 nodes).	Number of leaves $n_0 = n_2 + 1$.
Complete BT (CBT)	All levels filled except possibly last, which is filled left-to-right.	Array mapping has no gaps: Left is $\lfloor (2i+1) \rfloor$, Right is $\lfloor (2i+2) \rfloor$.
Perfect BT	All internal nodes have 2 children; all leaves at same depth.	Total nodes $\backslash (N = 2^{h+1} - 1)$; Total leaves = $\backslash (2^h)$.
Skewed BT	Every internal node has only 1 child (Left or Right skewed).	Degenerates to linked list; search degrades to $O(n)$.

Theorem: $n_0 = n_2 + 1$ (Leaves vs Degree-2 Nodes)

Proof: Let n = total nodes, B = total edges. Every node except root has 1 incoming edge $\Rightarrow B = n - 1$.

Also, edges originate from degree-1 and degree-2 nodes $\Rightarrow B = n_1 + 2n_2$.

Equating: $n - 1 = n_1 + 2n_2$. Substituting $n = n_0 + n_1 + n_2$ gives: $n_0 = n_2 + 1$. (Q.E.D.)

4.3 Binary Search Tree (BST) & Traversal Operations

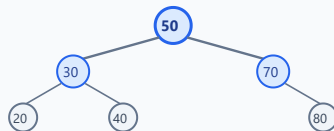
BST Invariant: For every node X , all keys in Left Subtree $< \text{key}(X) < \text{all keys in Right Subtree}$.

Tree Traversals:

1. **Inorder (L $\rightarrow$ Root $\rightarrow$ R):** Yields **strictly sorted ascending order!**
2. **Preorder (Root $\rightarrow$ L $\rightarrow$ R):** Used for tree cloning and prefix expressions.
3. **Postorder (L $\rightarrow$ R $\rightarrow$ Root):** Used for tree deletion & postfix evaluation.
4. **Level-Order (BFS):** Level-by-level exploration using a Queue.

BST Deletion (3 Cases):

- **Case 1 (Leaf Node):** Set parent's pointer to NULL, free node.
- **Case 2 (1 Child):** Link parent directly to child, bypass node.
- **Case 3 (2 Children):** Replace node value with **Inorder Successor** (minimum in right subtree) or **Inorder Predecessor**, then delete that successor node!



Inorder: 20 $\rightarrow$ 30 $\rightarrow$ 40 $\rightarrow$ 50 $\rightarrow$ 70 $\rightarrow$ 80 (Sorted!) **Preorder:** 50 $\rightarrow$ 30 $\rightarrow$ 20 $\rightarrow$ 40 $\rightarrow$ 70 $\rightarrow$ 80 **Postorder:** 20 $\rightarrow$ 40 $\rightarrow$ 30 $\rightarrow$ 80 $\rightarrow$ 70 $\rightarrow$ 50

FIGURE 4.1: BINARY SEARCH TREE (BST) ANATOMY & TRAVERSAL SEQUENCE ORDERS

4.4 Graph Representations & Handshaking Lemma

Adjacency Matrix [$V \times V$]: Space $O(V^2)$. Edge lookup $O(1)$. Optimal for dense graphs ($|E| \approx |V|^2$).
Adjacency List [V lists]: Space $O(V + E)$. Edge lookup $O(\deg(v))$. Optimal for sparse graphs ($|E| \ll |V|^2$).

Handshaking Lemma: $\sum \deg(v) = 2|E|$.
 The sum of degrees of all vertices is twice the number of edges. Consequently, the count of odd-degree vertices is always even!

4.5 Graph Traversals: Breadth-First Search (BFS) vs. Depth-First Search (DFS)

Metric	Breadth-First Search (BFS)	Depth-First Search (DFS)
Data Structure	Queue (FIFO) — Level-by-level expansion.	Stack (LIFO) or System Call Stack (Recursion).
Strategy	Visits all immediate neighbors before diving deeper.	Dives as deep as possible along each branch before backtracking.
Time Complexity	$O(V + E)$ (List) / $O(V^2)$ (Matrix).	$O(V + E)$ (List) / $O(V^2)$ (Matrix).
Space Complexity	$O(V)$ — Max frontier width stored in queue.	$O(V)$ — Max path depth stored on call stack.
Primary Uses	Shortest path in unweighted graphs; GPS routing.	Cycle detection; topological sort; maze solving.

5.1 Official MMDU Exam Blueprint & High-Frequency Questions

Paper Scheme (60 Marks | 3 Hours): Section A ($12 \times 1M = 12M$, compulsory) • Section B ($4 \times 2M = 8M$) • Section C ($4 \times 4M = 16M$) • Section D ($4 \times 6M = 24M$).

Q1: Why is Inorder traversal of BST sorted?

Ans: BST ordering dictates Left < Root < Right. Inorder recursively visits Left, then Root, then Right, visiting keys in strictly ascending order.

Q2: Why does Circular Queue eliminate False Overflow?

Ans: Modulo arithmetic ($(\text{rear}+1)\% \text{SIZE}$) wraps around to reuse vacant slots 0 to FRONT-1 freed up by previous dequeues.

5.2 Master Data Structure Complexity Comparison Matrix

Data Structure	Average Search	Worst Search	Average Insert	Average Delete	Worst Space	Primary Architectural Advantage
1D Array	$O(n)$	$O(n)$	$O(n)$ [End: $O(1)$]	$O(n)$ [End: $O(1)$]	$O(n)$	$O(1)$ random index access; cache locality.
Singly Linked List	$O(n)$	$O(n)$	Head: $O(1)$	Head: $O(1)$	$O(n)$	Dynamic sizing; $O(1)$ insert at head without shifting.
Doubly Linked List	$O(n)$	$O(n)$	Head/Tail: $O(1)$	$O(1)$ [given ptr]	$O(n)$	Bidirectional traversal; $O(1)$ node deletion.
Stack (Array/List)	$O(n)$	$O(n)$	$O(1)$ [Push]	$O(1)$ [Pop]	$O(n)$	Strict LIFO; manages nested/recursive states.
Circular Queue	$O(n)$	$O(n)$	$O(1)$ [Enqueue]	$O(1)$ [Dequeue]	$O(n)$	Strict FIFO; eliminates memory drift.
Binary Search Tree	$O(\log n)$	$O(n)$ [Skewed]	$O(\log n)$	$O(\log n)$	$O(n)$	Inorder gives sorted keys; logarithmic search.
Graph (Adj List)	BFS/DFS: $O(V + E)$		$O(1)$	$O(V + E)$	$O(V + E)$	Optimal memory for sparse networks; pathfinding.